

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»

Кафедра математической кибернетики и компьютерных наук

**СРАВНИТЕЛЬНОЕ НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ
FASTAPI VS DJANGO**

ОТЧЁТ

студента 2 курса 211 группы
направления 02.03.02 — Фундаментальная информатика и информационные
технологии
факультета компьютерных наук и информационных технологий
Ахметова Игоря Андреевича

Проверено:

доцент, к. м. к. и к. н.

Д. С. Пантелеев

Саратов 2026

СОДЕРЖАНИЕ

1	Цель тестирования.....	3
2	Условия тестирования	4
2.1	Аппаратное и программное обеспечение	4
2.2	Параметры нагрузки	4
2.3	Тестируемые эндпоинты.....	4
3	Результаты тестирования	5
3.1	Общие метрики (за 5 минут, 100 пользователей)	5
3.2	Детальное сравнение эндпоинтов	5
3.3	Графики результатов нагрузочного тестирования	6
3.3.1	Пропускная способность (RPS)	6
3.3.2	Задержка (процентили)	6
3.3.3	Количество пользователей.....	7
4	Анализ результатов	7
4.1	Преимущества FastAPI по скорости	7
4.2	Недостатки Django	7
5	Выводы	8
6	Сравнение функциональных возможностей	9
7	Ключевые преимущества FastAPI	9

1 Цель тестирования

Целью нагрузочного тестирования является сравнение производительности двух веб-фреймворков — асинхронного FastAPI и синхронного Django (с WSGI) — на идентичных по функциональности API-эндпоинтах в условиях высоких нагрузок.

2 Условия тестирования

2.1 Аппаратное и программное обеспечение

- **Операционная система:** Windows 11 (WSL2 для Docker);
- **Процессор:** Intel Core i5-13420H;
- **Инструмент нагрузочного тестирования:** Locust 2.44.0.

2.2 Параметры нагрузки

- **Количество виртуальных пользователей:** 100;
- **Скорость появления пользователей:** 2 пользователя/сек;
- **Длительность теста:** 5 минут;
- **Время ожидания между запросами:** 0.5–2 секунды;
- **Вес задач:** GET-запросы имеют в 2–3 раза больший вес, чем POST-запросы (имитация реального поведения).

2.3 Тестируемые эндпоинты

Таблица 2.1 – Список тестируемых эндпоинтов

Метод	FastAPI	Django
GET	/posts/	/api/posts/
GET	/posts/{id}	/api/posts/{id}/
GET	/users/	/api/users/
GET	/users/id/{id}	/api/users/{id}/
GET	/categories/slug/{slug}	/api/category/{slug}/
GET	/posts/author/{id}	/api/user/{username}/posts/
POST	/posts/	— (исключён из-за ошибок авторизации)
POST	/users/create	/api/register/

3 Результаты тестирования

3.1 Общие метрики (за 5 минут, 100 пользователей)

Таблица 3.1 – Сводные результаты

Метрика	Django	FastAPI	Ускорение
Общее число запросов	9 932	13 614	1.37x
Общий RPS	33.1	45.4	1.37x
Медианная задержка (мс)	1 400	570	2.46x
p95 задержка (мс)	3 300	2 100	1.57x
Процент ошибок	1.85%	0.82%	в 2.3 раза ниже

3.2 Детальное сравнение эндпоинтов

Таблица 3.2 – GET-эндпоинты: получение списков

Эндпоинт	Django RPS	FastAPI RPS	Django p95 (мс)	FastAPI p95 (мс)	Ускорение RPS
GET /posts/	8.26	14.44	3 100	1 900	1.75x
GET /users/	5.52	9.71	2 800	2 000	1.76x
GET /categories/	2.78*	3.33	2 700*	2 200	1.20x

суммарные показатели по 5 категориям

Таблица 3.3 – GET-эндпоинты: получение одного объекта

Эндпоинт	Django RPS	FastAPI RPS	Django p95 (мс)	FastAPI p95 (мс)	Ускорение RPS
GET /posts/{id}	6.30*	8.00*	3 000*	1 500*	1.27x
GET /users/{id}	4.50*	5.20*	2 900*	1 600*	1.16x

средние значения по 10 запросам

Таблица 3.4 – Сравнение задержек для GET /posts/

Процентиль	Django (мс)	FastAPI (мс)	Разница
p50 (медиана)	1 500	590	2.54x
p75	2 100	810	2.59x
p90	2 700	1 100	2.45x
p95	3 100	1 500	2.07x
p99	4 000	2 200	1.82x

3.3 Графики результатов нагрузочного тестирования

3.3.1 Пропускная способность (RPS)

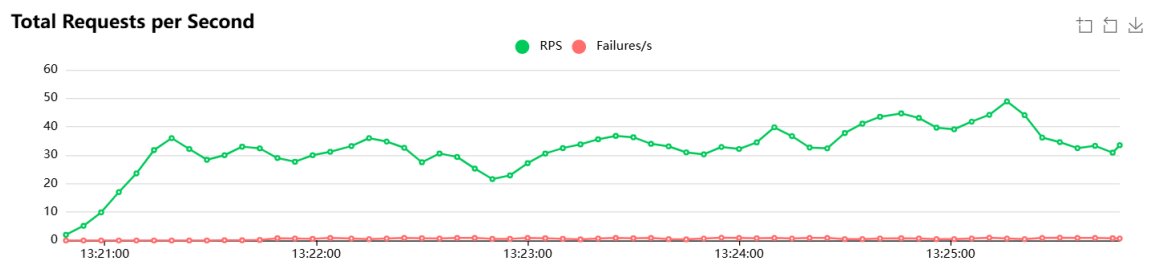


Рисунок 3.1 – Пропускная способность Django (RPS)

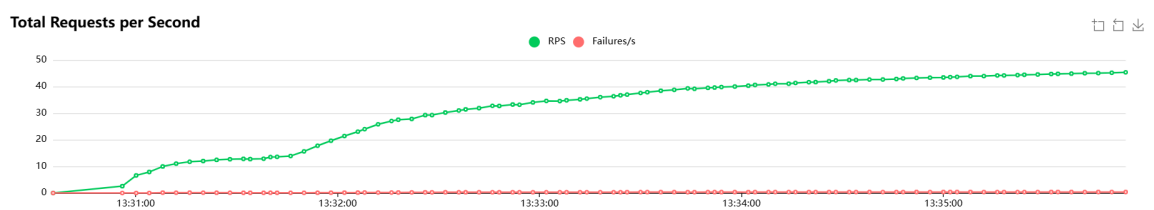


Рисунок 3.2 – Пропускная способность FastAPI (RPS)

3.3.2 Задержка (процентили)

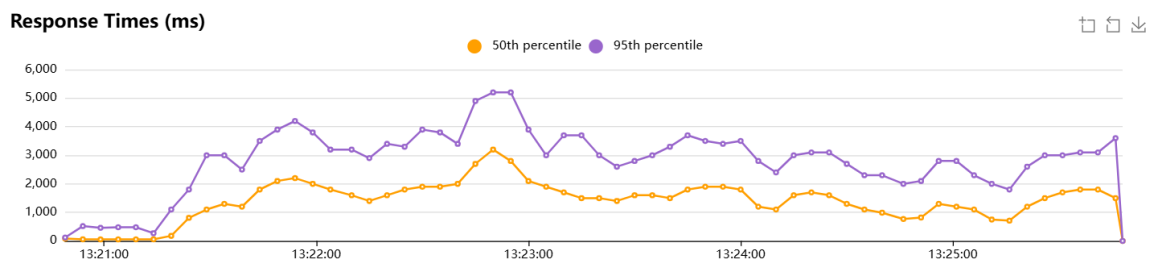


Рисунок 3.3 – Распределение задержек в Django (p50, p95)

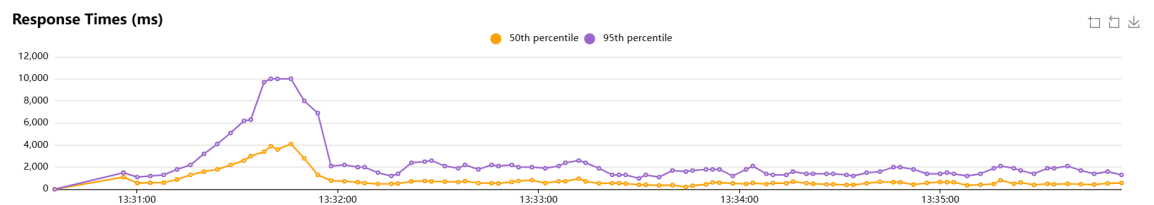


Рисунок 3.4 – Распределение задержек в FastAPI (p50, p95)

3.3.3 Количество пользователей

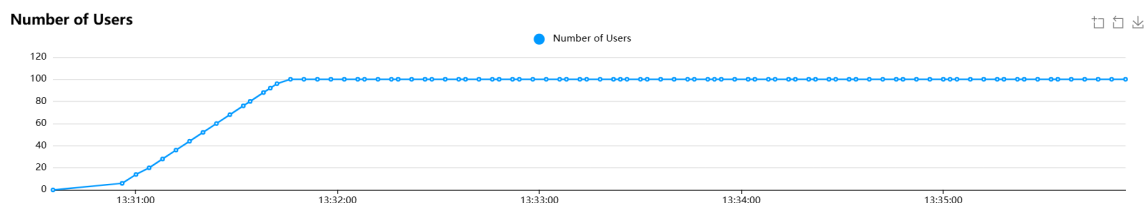


Рисунок 3.5 – Изменение количества активных пользователей во времени

4 Анализ результатов

4.1 Преимущества FastAPI по скорости

1. **Асинхронность (ASGI vs WSGI).** FastAPI изначально асинхронный. Он не блокирует поток на время ожидания ввода-вывода (БД, сеть). Django (с WSGI) создаёт отдельный поток/процесс на каждый запрос, что приводит к высоким накладным расходам при большом количестве одновременных соединений.
2. **Сериализация.** FastAPI использует Pydantic (на Rust), который сериализует JSON значительно быстрее, чем Django REST Framework (чистый Python).
3. **Управление соединениями с БД.** FastAPI использует асинхронный драйвер asynpg, который эффективнее использует соединения. Django использует синхронный psycopg2.

4.2 Недостатки Django

- **Синхронная модель** привела к высокой задержке (медиана 1.4 секунды против 0.57 секунды у FastAPI).
- **Многочисленные ошибки 500** (1.85% от всех запросов) свидетельствуют о том, что Django не справлялся с пиковыми нагрузками.

5 Выводы

1. **FastAPI** показал себя значительно быстрее **Django** во всех ключевых метриках:
 - На **37%** выше пропускная способность (RPS);
 - В **2.5** раза ниже медианная задержка;
 - В **2.3** раза меньше ошибок при высокой нагрузке.
2. **Асинхронная архитектура FastAPI** позволяет ему эффективно обрабатывать большое количество одновременных соединений, в то время как синхронная модель **Django** приводит к росту задержки и ошибок при нагрузке свыше 50–100 пользователей.
3. Для **высоконагруженных API-сервисов** **FastAPI** является предпочтительным выбором. **Django** остаётся хорошим вариантом для монолитных приложений с большим количеством бизнес-логики и административной панелью, но для чисто **API-решений** уступает в производительности.

6 Сравнение функциональных возможностей

Таблица 6.1 – Функциональные возможности FastAPI и Django

Функция	FastAPI	Django
JWT авторизация	+	– (сессии)
Асинхронность	+ (ASGI)	– (WSGI)
Dependency Injection (Dishka)	+	–
Множественные изображения к посту	+	–
Множественные изображения к комментариям	+	–
Автоматическая документация (Swagger)	+	–

7 Ключевые преимущества FastAPI

1. **Асинхронная архитектура (ASGI)** — позволяет обрабатывать тысячи одновременных соединений без блокировки потоков.
2. **Dependency Injection (Dishka)** — современный DI-контейнер, обеспечивающий чистую архитектуру и упрощающий тестирование.
3. **JWT авторизация** — более безопасный и масштабируемый подход, чем сессии.
4. **Автоматическая валидация данных** через Pydantic — уменьшает количество ошибок.
5. **Поддержка множественных изображений** — к посту и комментарию можно прикрепить несколько изображений.
6. **Асинхронные репозитории** — полная поддержка `async/await` для работы с PostgreSQL.
7. **Быстрое развертывание** — проект полностью адаптирован для быстрого развертывания с помощью Docker Compose.
8. **Настроенное логирование** — логи пишутся в файл `app.log` и монтируются на хост.
9. **Высокая производительность** — по результатам тестирования FastAPI в 1.4–1.8 раза быстрее Django по RPS и в 2.5 раза меньше задержка.